

Verificación Funcional de un Bus Serial Compartido con Arbitraje Distribuido

Luis Diego García Rojas Cristhian Lezcano Fernández
2020124283 2018111011

Instituto Tecnológico de Costa Rica, Escuela de Ingeniería Electrónica, Cartago, Costa Rica

1. Descripción del DUT

El dispositivo bajo prueba es el módulo `bs_gnrtr_n_rbtr`, que instancia una interfaz de bus (`bs_ntrfs_n_rbtr`) por cada terminal conectada y las une a través de tres líneas compartidas: la línea de datos serial (`bus`), la señal de bus ocupado (`bs_bsy`) y la señal de cambio de turno (`trn_chng`).

1.1. Señales

Cuadro 1: Señales de la interfaz del DUT, replicadas por terminal

Señal	Dirección	Descripción
<code>clk</code>	Entrada	Reloj del sistema
<code>reset</code>	Entrada	Reset activo alto
<code>pndng</code>	Entrada	La FIFO de salida del ambiente tiene datos
<code>D_pop</code>	Entrada	Paquete que la terminal desea transmitir
<code>pop</code>	Salida	El DUT consume el paquete de la FIFO de salida
<code>push</code>	Salida	El DUT entrega un paquete recibido
<code>D_push</code>	Salida	Paquete recibido desde el bus

1.2. Parámetros

Cuadro 2: Parámetros del DUT

Parámetro	Valor	Descripción
<code>bits</code>	1	Cantidad de buses instanciados
<code>drvrs</code>	4 ó 6	Terminales conectadas al bus
<code>pckg_sz</code>	16, 32, 64	Ancho del paquete en bits
<code>broadcast</code>	8'hFF	Identificador de difusión

El ambiente recibe estos mismos valores como parámetros, de modo que el ancho del paquete y la cantidad de terminales se fijan desde el script de corrida sin editar el testbench. Esa parametrización se aprovechó en ambas dimensiones: la batería completa de escenarios se ejecutó con seis terminales y paquetes de 16 bits, para que el arbitraje rotativo se ejercitara con más competidores por el bus, y el barrido de anchos se ejecutó con cuatro terminales. Cada corrida se analiza contra la configuración con la que fue obtenida.

1.3. Funcionamiento

- La línea de datos es común: solo una terminal transmite a la vez y el turno rota entre todas.

- El arbitraje es distribuido. Cada interfaz tiene un contador de turno propio (`Counter_arb`) que avanza con los flancos de `trn_chng`; la terminal cuyo contador coincide con su `ntrfs_id` obtiene la concesión del bus.
- Una terminal que tiene el turno pero no tiene nada que enviar cede el turno inmediatamente mediante la máquina de estados `Arbiter_st_Mchn`, sin ocupar el bus.
- La transmisión es serial: el paquete se serializa bit a bit con el módulo `serializer` y se deserializa del lado receptor.
- La dirección de destino viaja en los 8 bits más significativos del paquete. Cada interfaz compara esos bits contra su propio identificador y contra el valor de difusión; si coinciden, levanta `push` y entrega el paquete por `D_push`.
- Las FIFOs de entrada y salida no forman parte del DUT: las emula el ambiente de verificación a través del protocolo `pndng/pop/D_pop`.

1.4. Formato del paquete

Cuadro 3: Formato del paquete para los anchos soportados

Ancho	Dirección	Contenido
16 bits	bits 15:8	bits 7:0
32 bits	bits 31:24	bits 23:0
64 bits	bits 63:56	bits 55:0

Con `drvrs` terminales conectadas, los identificadores válidos son 0 a `drvrs-1`, el identificador de difusión es 255 y el resto del rango corresponde a direcciones que no pertenecen a ninguna terminal.

2. Plan de pruebas

Los casos con prefijo TG corresponden a casos generales y los de prefijo TE a casos de esquina. Los rangos de aleatorización y los criterios de aprobación completos están en el documento de test plan entregado previamente; la Tabla 4 resume cada caso y el escenario del ambiente que lo ejerce.

Cuadro 4: Casos del plan de pruebas y escenario que los ejerce

Caso	Descripción	Escenario
TG-01	Transmisión simple y secuencial	GENERAL
TG-02	Anchos de paquete 16/32/64	GENERAL $\times$ 3
TG-03	Rotación del turno	GENERAL
TE-04	Transmisión broadcast	BCAST
TE-05	Acceso simultáneo	OVERFLOW
TE-06	Direcciones inválidas	INVALID
TE-07	FIFO de salida llena	OVERFLOW
TE-08	FIFO de salida vacía	GENERAL
TE-09	Envío a sí mismo	SELF

El caso de underflow planteado originalmente como TE-05 se replanteó durante la implementación. Como las FIFOs son emuladas por el ambiente y no forman parte del DUT, no existe una condición de underflow interna del dispositivo que verificar; lo que sí tiene sentido comprobar es que el DUT nunca solicite un `pop` con la cola vacía, es decir, que respete el protocolo de `pnrng`. El emulador de FIFO lleva un contador de esa violación (`n_underflow`) que se reporta en todas las corridas, de modo que el caso se verifica de forma continua en lugar de como escenario aparte.

Dos casos del plan quedaron sin implementar y se declaran como tales. El primero es TG-03, el reset del sistema en caliente: el reset solo se aplica al arranque desde el testbench, de modo que no se verificó el retorno a un estado conocido con un paquete en vuelo o durante un cambio de turno. El segundo es TE-06, la sincronía de los registros de turno, que requiere observar señales internas de cada interfaz y no es accesible desde el flujo de transacciones del ambiente. Ambos quedan como trabajo pendiente y se retoman en las conclusiones.

3. Ambiente de verificación

El ambiente sigue la arquitectura por capas vista en clase. Cada unidad corre en su propio proceso, lanzado con `fork ... join_none` desde la clase `bus_env`, y la comunicación entre unidades se realiza exclusivamente por mailboxes.

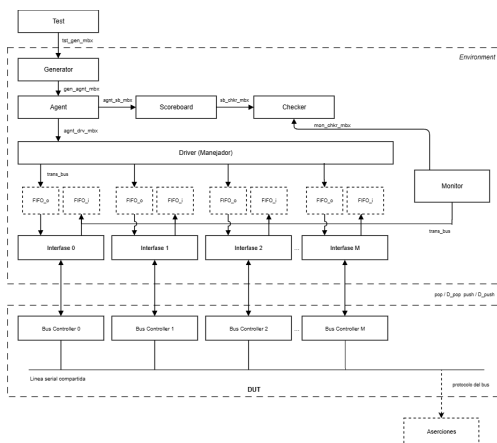


Figura 1: Diagrama del ambiente de verificación.

3.1. Mailboxes y paquetes de comunicación

Cuadro 5: Paquetes de transacción del ambiente

Pck	Tipo / Mailbox	Origen → Destino
pck1	transaction / mbx_gen_agent	Generator → Agent
pck2	transaction / mbx_agent_drv	Agent → Driver
pck3	transaction / mbx_agent_sb	Agent → Scoreboard
pck4	expected_item / mbx_sb_chk	Scoreboard → Checker
pck5	transaction / mbx_mon_chk	Monitor → Checker

Dentro del Driver y del Monitor existe además un mailbox por terminal que comunica al proceso padre con cada proceso hijo (`parent_child_mbx` y `child_parent_mbx`).

3.2. Campos del objeto de transacción

Cuadro 6: Campos de la clase `transaction`

Campo	Descripción
<code>src_terminal</code>	Terminal que transmite (aleatorizado)
<code>dst_addr</code>	Dirección de destino (aleatorizada)
<code>kind</code>	Unicast, broadcast o dirección inválida
<code>payload</code>	Contenido del paquete (aleatorizado)
<code>delay</code>	Ciclos de espera antes de entrar a la FIFO
<code>rx_terminal</code>	Terminal donde el monitor observó el paquete
<code>sent_time</code>	Instante del <code>pop</code> que lo sacó de la FIFO
<code>receive_time</code>	Instante del <code>push</code> observado

Los dos últimos campos se inicializan en `-1` para distinguir “todavía no ocurrió” de “ocurrió en el tiempo cero”, y son los que alimentan directamente las columnas del reporte CSV.

3.3. Estructura padre/hijos

Tanto el Driver como el Monitor siguen el esquema de un proceso padre y un proceso hijo por terminal exigido en los lineamientos:

- El **Driver** recibe cada transacción del Agent y la encamina al mailbox del hijo correspondiente a su terminal de origen. Cada hijo es un `fifo_emulator` que mantiene su propia cola (`pkt_queue`) y corre a su vez dos procesos: uno que aplica el retardo del paquete antes de ingresarlo a la cola, y otro que atiende el protocolo del DUT ciclo a ciclo.
- El **Monitor** lanza un `mon_child` por terminal que observa `push` en cada flanco y reconstruye el paquete a partir de `D_push`; el padre recoge lo observado por todos los hijos y lo reenvía al Checker por un único mailbox.

4. Implementación del ambiente

El ambiente está compuesto por los siguientes archivos:

- `interface.sv`: interfaz y clocking blocks.
- `transaction.sv`: objeto de transacción y restricciones.
- `generator.sv`: generación aleatorizada de estímulos.
- `agent.sv`: reparto hacia Driver y Scoreboard.
- `driver.sv`: emulador de FIFO y driver padre.
- `monitor.sv`: hijos observadores y monitor padre.
- `scoreboard.sv`: modelo de referencia del enrutamiento.
- `checker.sv`: comparación, estadísticas y reporte CSV.
- `env.sv`: instanciación, conexión y vaciado.
- `test.sv`: configuración del escenario y veredicto.
- `testbench.sv`: reloj, reset, DUT e interfaz.

4.1. Interfaz

La interfaz `dut_compl_if` agrupa todas las señales del DUT y define dos clocking blocks. El del driver (`cb_drv`) maneja `pndng` y `D_pop` con una salida desplazada 1 ns respecto al flanco, de modo que el DUT nunca observa una carrera entre el estímulo y el reloj; el del monitor (`cb_mon`) es completamente pasivo y solo muestrea.

4.2. Clase transaction

Describe un paquete del bus. Aleatoriza la terminal de origen, el tipo de destino, la dirección, el contenido y el retardo entre mensajes. El tipo de destino se sortea primero con una distribución configurable (`pct_bcast`, `pct_inv`) y la dirección se restringe después según ese tipo: un identificador de terminal para unicast, el valor de difusión para broadcast, y cualquier dirección fuera de ambos conjuntos para el caso inválido. Dentro del rango inválido se da peso adicional a la primera y a la última dirección no asignada, por ser los bordes del rango.

Dos restricciones merecen mención aparte. La primera impide que una terminal se dirija a sí misma, porque mientras transmite mantiene el bus y por construcción nunca lee su propio paquete. La segunda, activada por la bandera `force_self`, invierte esa condición para el caso dirigido en el que todos los paquetes se dirigen a su propio origen.

Las funciones `pack()` y `unpack()` concentran el formato del paquete en un solo lugar: la primera arma la palabra que el DUT lee de `D_pop` y la segunda reconstruye los campos a partir de lo observado en `D_push`.

4.3. Clase generator

Crea las transacciones y las entrega al Agent. Opera en tres modos según la configuración del test: reparto por terminal, donde cada uno sortea su propia cantidad de transacciones dentro de un rango; cantidad total fija con origen aleatorio; e inundación dirigida, donde todas las transacciones provienen de una misma terminal. Al terminar levanta la bandera `done`, que el ambiente usa para iniciar el vaciado.

4.4. Clase bus_agent

Recibe cada transacción del generator y coloca el mismo handle en los mailboxes del Driver y del Scoreboard. Compartir el handle en lugar de copiarlo es deliberado: el Driver escribe `sent_time` sobre el objeto y el Checker lee ese mismo valor a través de la expectativa que recibe del Scoreboard, sin necesidad de un mecanismo adicional de sincronización.

4.5. Clase fifo_emulator

Emula la FIFO de salida de una terminal. El proceso `feed()` toma los paquetes del padre, espera el retardo de cada uno y los encola; si la cola está en su capacidad máxima, el paquete espera a que el DUT libere un espacio, lo que reproduce el comportamiento de contrapresión de un dispositivo real en lugar de perder datos. El proceso `drive()` atiende el protocolo: ante un `pop` retira el paquete de la cabeza y le registra el `sent_time`, y luego presenta la nueva cabeza levantando `pndng`. Lleva además las estadísticas de ocupación máxima, veces que la cola se llenó, y solicitudes de `pop` recibidas con la cola vacía.

4.6. Clase bus_driver

Instancia un emulador por terminal, los arranca y encamina cada transacción al que corresponda según su origen. Expone `is_empty()`, que el ambiente consulta para saber si ya se entregaron todos los paquetes al DUT, y la suma de violaciones de protocolo de todos los hijos.

4.7. Clases mon_child y bus_monitor

El hijo observa su terminal en cada flanco y, cuando `push` está activo y el sistema no está en reset, reconstruye el paquete, le anota la terminal receptora y el instante de recepción, y lo envía al padre. El padre lanza los hijos en procesos independientes y reenvía todo al Checker. El índice del lazo se declara `automatic` antes del `fork`; de lo contrario todos los hijos compartirían la misma variable y terminarían observando la última terminal.

4.8. Clase bus_scoreboard

Es el modelo de referencia del enrutamiento. Para cada transacción clasifica el destino en cuatro cate-

gorías y emite las expectativas correspondientes: una sola para unicast, una por cada terminal distinta del emisor para broadcast, y ninguna para direcciones inválidas o para paquetes dirigidos al propio emisor. Lleva además el conteo por categoría que aparece en el reporte final.

4.9. Clase bus_checker

Compara lo observado contra lo esperado y mantiene una cola de expectativas pendientes por terminal receptora. El emparejamiento no puede hacerse solo por contenido: con paquetes de 16 bits el campo de datos es de 8 bits, así que dos transacciones distintas pueden producir exactamente la misma palabra. Para resolverlo solo se consideran candidatas las expectativas cuyo paquete ya salió del ambiente, y entre ellas se escoge la que se transmitió primero.

Sobre esa base reporta tres clases de error: paquetes observados que nadie esperaba, paquetes entregados fuera del orden en que su terminal los transmitió, y expectativas que quedaron sin resolver al final de la simulación. Acumula también el retardo mínimo, promedio y máximo, y escribe el reporte CSV.

4.10. Clase bus_env

Instancia los componentes, crea los mailboxes y lanza cada unidad en su propio proceso. Su tarea `wait_drain()` resuelve el final de la simulación: espera a que el Driver haya entregado todo, a que el Checker no tenga expectativas pendientes y a un margen adicional proporcional al ancho del paquete, todo ello contra un temporizador de seguridad calculado a partir del tiempo que ocupa cada paquete en el bus multiplicado por la cantidad de terminales en contención. Si ese temporizador vence primero, se reporta un error de vaciado.

4.11. Verificación del protocolo de la interfaz

Además de la comparación entre lo esperado y lo observado, el ambiente vigila de forma continua el protocolo de las señales de la interfaz, en los dos extremos del camino de un paquete.

En el lado de la transmisión, el emulador de FIFO comprueba cada ciclo que el DUT no solicite un `pop` sobre una terminal cuya cola está vacía. Esa es la condición que el protocolo de `pndng` debe impedir: si el DUT la violara, retiraría un paquete inexistente y el dato transmitido al bus sería basura. La violación se contabiliza por terminal en `n_underflow` y se reporta al final de cada corrida, de modo que la comprobación está activa en todos los escenarios y no solo en uno dirigido.

En el lado de la recepción, cada hijo del monitor captura `D_push` únicamente en los ciclos en que `push` está activo y el sistema no está en reset. Que la cantidad de capturas coincida exactamente con la cantidad

de recepciones que predice el modelo de referencia, sin capturas sobrantes, es la comprobación de que el DUT no levanta `push` fuera de una entrega legítima.

Estas dos comprobaciones cumplen el papel de las aserciones dentro del ambiente: vigilan el protocolo de la interfaz en cada ciclo, de forma independiente del escenario que se esté ejecutando, y no solo el resultado final de la transacción. Están implementadas de forma procedural dentro del emulador de FIFO y del hijo del monitor, en lugar de como propiedades SVA concurrentes, pero cubren las dos propiedades que importan en los extremos del camino de un paquete: que el DUT no retire datos que no existen y que no entregue datos que nadie envió. Su resultado se reporta en todas las corridas mediante los contadores `n_underflow` por terminal y la comparación exacta entre capturas y recepciones esperadas, y en ninguno de los escenarios ejecutados se registró una violación.

4.12. Clase bus_test

Selecciona el escenario con el plusarg `+TEST_CASE`, ajusta las perillas del generador y emite el veredicto a partir de los contadores del Checker. Admite además `+NUM_TX`, `+FIFO_DEPTH` y `+ntb_random_seed`. El veredicto contempla el caso particular de los escenarios en los que no debe recibirse ningún paquete: cero recepciones esperadas y cero observadas es el resultado correcto para `INVALID` y `SELF`.

5. Proceso de compilación y simulación

La compilación y las corridas se automatizan con el script `run.sh`, que compila una sola vez y luego ejecuta cada escenario sobre el mismo binario con una semilla común. El ancho de paquete, la cantidad de terminales, la semilla y la lista de casos se fijan por variables de entorno, sin editar el testbench.

```
vcs -Mupdate -sverilog -full64 \
testbench.sv -o salida \
-pvalue+testbench.p_width=$WIDTH \
-pvalue+testbench.p_drvs=$DRVS \
-kdb -lca -debug_acc+all \
-l comp.log +lint=TFIPC-L \
-cm line+tgl+cond+fsm+branch+assert
```

Figura 2: Comando de compilación.

```
./salida -cm line+tgl+cond+fsm+branch+assert \
+ntb_random_seed=$SEED \
+TEST_CASE=$case_name +NUM_TX=0 \
-l run_${case_name}_w${WIDTH}_seed${SEED}.log
```

Figura 3: Ejecución de un escenario con semilla fija.

Fijar la semilla hace cada corrida reproducible: con el mismo valor, la secuencia de transacciones generada es idéntica entre ejecuciones, lo que permite comparar

escenarios entre sí sabiendo que las terminales reciben la misma carga. Al terminar, el script recorre los CSV producidos y genera un histograma por cada uno, saltándose los que no tienen recepciones.

6. Resultados

La batería de escenarios se ejecutó con paquetes de 16 bits, seis terminales y semilla 1234; el barrido de anchos se ejecutó por separado y se presenta al final de esta sección. La Tabla 7 resume los cinco escenarios. Todas las corridas aprobaron, sin paquetes inesperados, sin inversiones de orden, sin pérdidas y sin violaciones del protocolo de `pdng`.

Cuadro 7: Resumen de la batería de escenarios (w=16, 6 terminales, semilla 1234)

Caso	Tx	Esp.	OK	Err.	Retardo prom.
GENERAL	85	127	127	0	913,23 ns
BCAST	85	425	425	0	920,94 ns
INVALID	85	0	0	0	—
SELF	85	0	0	0	—
OVERFLOW	20	20	20	0	260,50 ns

6.1. Caso general

El generador reparte entre 5 y 20 transacciones por terminal, con una mezcla de tráfico unicast, de difusión y hacia direcciones inexistentes. En esta corrida produjo 85 transacciones en total.

```
[GEN] Terminal 0: 18 transacciones
[GEN 0] origen=0 destino=1 retardo=7 dato=5c
[GEN 1] origen=0 destino=5 retardo=1 dato=40
[GEN 3] origen=0 destino=255 retardo=7 dato=fa
[GEN 11] origen=0 destino=92 retardo=9 dato=da
```

Figura 4: Transacciones de los tres tipos de destino emitidas por el generador.

El Scoreboard clasificó esas transacciones y calculó las recepciones que debían observarse: cada paquete unicast genera una, cada broadcast genera una por cada terminal distinta del emisor, y los dirigidos a direcciones inexistentes no generan ninguna.

```
[SB] unicast=57 broadcast=14 invalidos=14
a_si_mismo=0 recepciones_esperadas=127
[CHK] ok=127 unexpected=0 out_of_order=0 missing=0
[CHK] retardo min=190.00 prom=913.23 max=1170.00 (ns)
[TEST] RESULTADO: PASS (127 paquetes correctos)
```

Figura 5: Reporte final del caso general.

Las 127 recepciones esperadas se observaron exactamente, sin paquetes inesperados, sin inversiones de orden y sin pérdidas. El reparto del tráfico entre las seis terminales y la ausencia de paquetes perdidos son la evidencia de que la rotación del turno funcionó: ninguna terminal quedó sin acceso al bus a pesar de que

la carga fue desigual, entre 6 y 20 transacciones según la terminal.

6.2. Transmisión broadcast

Con todo el tráfico dirigido al identificador de difusión, las mismas 85 transacciones generan 425 recepciones esperadas, cinco por paquete, que es la cantidad de terminales menos el emisor.

```
[SB] unicast=0 broadcast=85 invalidos=0
a_si_mismo=0 recepciones_esperadas=425
[CHK] ok=425 unexpected=0 out_of_order=0 missing=0
[CHK] retardo min=190.00 prom=920.94 max=1170.00 (ns)
```

Figura 6: Reporte final del caso de difusión.

Todas las copias llegaron. El retardo promedio es prácticamente el mismo que en el caso general, lo que confirma que un paquete de difusión ocupa el bus una sola vez: la replicación ocurre en los receptores y no cuesta turnos adicionales.

6.3. Direcciones inválidas

Con todo el tráfico dirigido a direcciones que no corresponden a ninguna terminal, el modelo de referencia no genera ninguna expectativa y ninguna terminal debe aceptar los paquetes.

```
[SB] unicast=0 broadcast=0 invalidos=85
a_si_mismo=0 recepciones_esperadas=0
[CHK] ok=0 unexpected=0 out_of_order=0 missing=0
[TEST] RESULTADO: PASS (0 paquetes esperados, 0
recibidos: esperado para INVALID y SELF)
```

Figura 7: Reporte final del caso de direcciones inválidas.

El resultado relevante no es solo que nadie aceptó los paquetes, sino que la simulación terminó en el mismo tiempo que el caso general. Los mensajes consumieron su turno de bus y el turno siguió rotando con normalidad: un destino inexistente no bloquea el bus.

6.4. Envío a sí mismo

Este escenario fuerza que cada terminal se dirija a sí misma. El modelo de referencia no espera ninguna recepción, porque la terminal que transmite mantiene el bus durante toda la transferencia y no lee su propia palabra.

```
[SB] unicast=0 broadcast=0 invalidos=0
a_si_mismo=85 recepciones_esperadas=0
[CHK] ok=0 unexpected=0 out_of_order=0 missing=0
```

Figura 8: Reporte final del caso de envío a sí mismo.

El DUT se comportó como predice el modelo: ningún paquete fue aceptado por su propio emisor y ninguna otra terminal lo recibió por error.

6.5. FIFO de salida llena

Para forzar la condición de cola llena se inunda una sola terminal con 20 paquetes consecutivos sin retardo entre ellos y se reduce la profundidad de la FIFO emulada a dos entradas.

```
[GEN] Modo overflow: inundando terminal 0 con
      20 transacciones
[FIFO_EMU] Terminal 0: FIFO llena (2/2),
      esperando espacio (overflow forzado)
...
[DRV] Terminal 0: ocupacion maxima=2,
      veces que se lleno (stall)=18, underflow=0
[CHK] ok=20 unexpected=0 out_of_order=0 missing=0
[CHK] retardo min=260.00 prom=260.50 max=270.00 (ns)
```

Figura 9: Detección de la condición de FIFO llena.

La cola se llenó 18 veces y en todas ellas el paquete esperó espacio en lugar de perderse: los 20 paquetes llegaron a su destino y en el orden correcto. El retardo promedio cae a una tercera parte del caso general porque una sola terminal está compitiendo por el bus, de modo que no espera turnos ajenos.

6.6. Anchos de paquete

Para verificar el soporte de los tres anchos se repitió el escenario general con paquetes de 32 y de 64 bits, esta vez con cuatro terminales y la misma semilla. La Tabla 8 compara las tres corridas.

Cuadro 8: Escenario general en los tres anchos de paquete

Ancho	Term.	Esp.	OK	Mín.	Prom.	Máx.
16	6	127	127	190	913,2	1170
32	4	61	61	350	1089,7	1410
64	4	61	61	670	2075,9	2690

El contenido se preservó íntegro en los tres anchos y la dirección se interpretó correctamente en los 8 bits más significativos en todos los casos: las 61 recepciones esperadas se observaron exactamente tanto en 32 como en 64 bits, con la misma clasificación de tráfico (31 unicast, 10 de difusión y 6 hacia direcciones inexistentes).

Las corridas de 32 y 64 bits comparten configuración y semilla, de modo que son directamente comparables entre sí: generan las mismas 47 transacciones con los mismos destinos y retardos, y la única diferencia es el ancho del paquete. Al duplicar el ancho, los tres estadísticos de retardo crecen por un factor prácticamente idéntico de 1,91. Que el factor sea cercano a dos confirma que el tiempo dominante es la ocupación serial del bus, proporcional a la cantidad de bits transmitidos; que quede por debajo de dos revela un componente fijo por transferencia, del orden de 100 ns, que no escala con el ancho y que corresponde al protocolo de concesión y cambio de turno.

La corrida de 16 bits no es comparable con las otras dos, porque se ejecutó con seis terminales en lugar de cuatro: su retardo promedio refleja una competencia

por el bus mayor, con dos emisores adicionales esperando turno.

6.7. Reporte de paquetes en formato CSV

El Checker escribe un archivo por escenario, nombrado con el caso, el ancho de paquete y la semilla, con una fila por paquete entregado y las columnas exigidas en el enunciado.

Cuadro 9: Columnas del reporte CSV

Columna	Descripción
Tiempo_Envio	Instante en que el DUT retiró el paquete
Terminal_Origen	Terminal que lo transmitió
Terminal_Destino	Terminal que lo recibió
Tiempo_Recibido	Instante del push observado
Retardo	Diferencia entre ambos instantes

6.8. Histograma de tiempos de retardo

El script de GNUplot calcula el ancho de cada barra a partir de los datos reales y lo redondea a un múltiplo del período del reloj, de modo que las barras quedan alineadas con los retardos efectivamente observados para cualquier ancho de paquete.

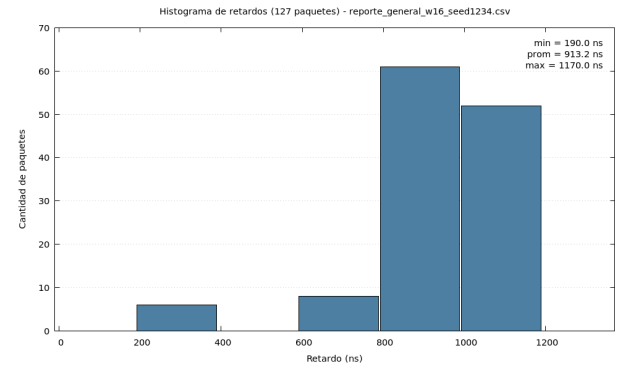


Figura 10: Retardo de los paquetes recibidos, caso general.

El rango observado va de 190 ns a 1170 ns, con un promedio de 913,2 ns sobre los 127 paquetes entregados. La distribución no es uniforme: 113 de los 127 paquetes, un 89 %, se concentran por encima de los 770 ns, mientras que solo 6 quedan en el extremo inferior. Esa concentración es consecuencia directa del arbitraje rotativo con seis terminales activas. El límite inferior corresponde a un paquete que encontró su turno disponible casi de inmediato, situación poco frecuente cuando todas las terminales tienen tráfico pendiente; el grueso de los paquetes espera a que el turno complete una vuelta, y el límite superior de 1170 ns corresponde a la espera máxima, con las cinco terminales restantes transmitiendo antes. La diferencia entre ambos extremos es, en esencia, el costo del arbitraje bajo carga, y el hecho de que exista un máximo acotado y no una cola larga es la evidencia de que ninguna terminal sufre inanición.

6.9. Hallazgos sobre el DUT

Ninguno de los escenarios ejecutados produjo errores funcionales. Se documentan dos comportamientos que, sin ser fallas, conviene dejar explícitos porque condicionan cómo debe construirse el modelo de referencia:

- Una terminal nunca recibe un paquete dirigido a sí misma. Mientras transmite mantiene el bus y su lógica de recepción no compara la palabra que ella misma está emitiendo.
- Un paquete de difusión llega a todas las terminales excepto al emisor, por la misma razón. El modelo de referencia debe emitir `drvrs-1` expectativas por cada broadcast, no `drvrs`.